

SEQ2SEQ ve ATTENTION MEKANİZMASI

Seq2Seq (sequence to sequence), bir veri dizisini yani girdisini alıp başka bir veri dizisine yani çıktı halinde dönüştürmek için kullanılan bir öğrenme mimarisidir.

RNN/LSTM bu mimariyi hayata geçirmek için kullanılan motrolardır. Bu mimariyi inşa edebilmek için verileri sırasıyla işleyebilen bir yapıya ihtiyaç duyar. Burda devreye RNN (Recurrent Neural Networks/ Tekrarlayan Sinir Ağı) kullanılır veya daha gelişmiş olan LSTM (Long Short-Term Memory/ Uzun Kısa Süreli Bellek Mimarisi) girer.

RNN cümle uzadığında cümlenin başındaki bilgiyi unutmaya başlar. LSTM içinde bulunan kapı ('gate') mekanizmaları sayesinde hangi bilginin daha önemli olduğunu ve daha uzun süre saklanması gerektiğini anlar.

Seq2seq için Encoder-Decoder mimarisi veya Bağlantı vektörü yetmez. Attention mekanizması ve Eğitim (Teacher Forcing, Beam Search, Greedy Search) ve Tahmin stratejileride kullanılır.

Seq2seq mimarisin daha iyi anlamak için 2 ana çerçeve şeklinde ve aradaki köprüye odaklanmamız gerekir.

Bunlar Endcoder-Decoder ve Bağlantı Vektörü (Context Vektörü)dür;

ENCODER (Kodlayıcı)

Encoder kısmı genellikle bir dizi RNN/LSTM hücresinden oluşur ve girdi dizesindeki her bir kelimeyi sırasıyla okur, her kelimeyi okuduğunda o ana kadar öğrendiği her bir bilgiyi sayısal biçimde gizli durum notu tutar.

GİZLİ DURUM (Hidden State)

Gizli durum bir anlık bellek gibi çalışır. Kelimeler okunurken sırasıyla gizli duruma kaydedilir. Örn: 'The black cat come.' Cümlesinde ilk başta 'The' kelimesi okunur. Bu kelime sayısal bir vektöre dönüşür ve belleğe kaydedilir. Sonrasında 'Black' sözcüğüne geçilir. 'Black' kelimeside sayısal vektöre dönüştürülür ve aynı şekilde gizli durum kısmına kaydedilir.

Burada bu iki vektör matematiksel olarak (çarpım, toplama veya filtreme) harmanlanır. Bu en sonki kelimeye kadar devam eder.

Not:Kelimeler kendileri vektör haline dönüştürülmez, o kelimenin birden fazla anlamı varsa bu karmaşıklılığa yol açar. Bu yüzden kelime anlam yumağı şeklinde vekörlere dönüştürülür.(Örn:’Come’ kelimesi bir yürüme eylemi biçiminde ifade edilerek vektöre dönüştürülür.)

BAĞLAM VEKTÖRÜ (Context Vector)

Gizli durumda güncellenen en sonki nota Bağlam Vektörü denir. Bu vektör cümlelerin bütün anlamını içeren sıkıştırılmış bir dosya gibidir. Encoder ve Decoder arasındaki tek bağlantı kısmıdır.

Decoderın ilk kelimeyi üretmesi için gereken bir yakıt gibidir.

DECODER (Çözücü)

Decoder da yine RNN/LSTM hücrelerini kullanarak çıktığı adım adım üretir. İlk olarak bir başlangıç işareti alır, bağlam vektörüne bakar ve ilk kelimeyi tahmin eder.

Kelimeyi tahmin etmesinin sebebi bağlam vektöründe kelimenin kendisi değil anlam yumağıdır. Decoder bu anlam yumağını çözerek kendi dilindeki en yakın kelimeyi tahmin etmek zorundadır. Bunun sebebi çeviri 1’e 1 bir eşleşme değil bir olasılık hesabıdır.

ATTENTION (Dikkat) MEKANİZMASI

Seq2seq sistemlerinin en önemli parçasıdır. Sadece Encoder-Decoder kullanıldığında, tüm cümle bir vektöre sıkıştırılır ve bu da ‘bilgi darboğazına’ sebep olur. Attention mekanizması cümlelerin her bir kelimesine ayrı ayrı ve dinamik olarak odaklanabilir.

Yapay zeka her çıktı adımında girdi cümlesinin tamamını tarar ve o anki çeviri için en önemli olan kelimelere daha fazla ağırlık verir. Birden fazla kelimelere ağırlık verebilir.

(Örn:’The black cat come’ cümlesinde ‘kedi’ kelimesi hem ‘cat’ hemde ‘black’ sözcüklerinin ağırlık verilmesiyle bulunur.)

Decoder'ın o anki durumu ile Encoder'daki tüm durumlar arasında bir noktasal çarpım veya benzerlik hesabı yapılır.

Eğer matematiksel olarak iki vektör birbirine "yakınsa" puan yüksek çıkar. Bu puanlar **Softmax** fonksiyonuna sokulur ve toplamı 1 olan olasılıklara (ağırlıklara) dönüşür. (Örn; 'kedi' çevrilecekse, girdi cümlesindeki 'cat' vektörüyle benzerlik tavan yapar ve ağırlık ona verilir.)

EĞİTİM ve TAHMİN STRATEJİLERİ

TEACHER FORCİNG

Normal bir eğitimde modelin yaptığı hatalar bir sonraki adımı etkilerken, Teacher Forcing bu zincirleme hatayı kırarak 'öğretmen' gibi doğru cevabı sisteme doğrudan verir. İhtiyaç duyulmasının ana sebebi bu hatanın zincirleme etkisi yaratmamasını istememizdir.

Geleneksel RNN/LSTM eğitiminde, model bir kelimeyi tahmin eder ve bu tahmini bir sonraki adımın girdisi olarak kullanır.

Eğer model ilk adımda yanlış bir kelime seçerse (örneğin 'Cat' yerine 'Köpek' derse), sonraki tüm kelime tahminleri bu yanlış üzerine inşa edilir.

ÇALIŞMA MANTIĞI

Teacher Forcing devredeyken, modelin her adımda ürettiği çıktı (doğru ya da yanlış fark etmeksizin) bir kenara bırakılır. Bir sonraki adımın girdisi olarak modelin kendi tahmini değil, veri setindeki gerçek kelime verilir.

'The Black Cat Come' çevirisinde model ilk adımda 'Beyaz' dediyse, sistem "Hayır, doğrusu Siyah" der ve bir sonraki adımda modelin önüne sanki 'Siyah' demiş gibi doğru veriyi koyar.

BEAM SEARCH

Model kelime kelime çeviri yaparken, her adımda sadece o anki en yüksek olasılıklı tek bir kelimeyi seçerse buna "Greedy Search" denir. Ve an seçilen en iyi kelime, cümlelerin devamını felakete sürükleyebilir.

Beam Search, sadece tek bir yola sapmak yerine, aynı anda en olası **k** adet sayıya farklı yolu hafızasında tutar.

İlk adımda en yüksek olasılığa sahip **k** adet kelimeyi seçer. İkinci adımda bu 3 kelimenin her biri için olası tüm sonraki kelimeleri hesaplar. Artık elinde çok fazla olasılık vardır. Sistem, her yolun toplam olasılığını hesaplar ve yine sadece en iyi 3 yolu tutar, diğerlerini siler. Cümlelerin sonuna gelindiğinde, bu 3 yoldan toplam puanı en yüksek olanı nihai çeviri olarak seçer.

K yı eğer çok düşük bir değer seçilirse örneğin 1, bu durum Greedy Search olur ve bu bizim istemediğimiz bir durum. Eğer **k** yı çok büyük seçersek bu durumda çok fazla olasılık olmuş olur ama bu durumda bilgisayırı çok fazla yormuş olur. Genel olarak 5-10 arası bir değer yeterli ve uygundur.

BLEU (Bilingual Evaluation Understudy) SKORU

BLEU, makine tarafından üretilen bir çevirinin, insan tarafından yapılmış referans çeviriye ne kadar benzediğini ölçen matematiksel bir metriktir. Çeviri kalitesini ölçmek için binlerce çeviriye insana okutmak çok yavaş ve pahalıdır. BLEU, bu süreci saniyeler içinde yapan objektif bir puanlama sistemi sunar. Puan aralığı 0 ile 1 arasındadır. 1'e yaklaşması, makine çıktısının insan çevirisiyle yüksek oranda örtüştüğünü gösterir.

N-GRAM HASSASİYETİ

'n-gram' basitçe, bir cümledeki ardışık n adet gruptan oluşan kelimeler demektir.

- **1-gram (unigram):** Kelimelerin tek tek doğruluğuna bakar ("kedi", "siyah").
- **2-gram (bigram):** Kelime çiftlerinin doğruluğuna bakar ("siyah kedi").
- **3-gram (trigram):** Üçlü grupların akışına bakar ("siyah kedi oturdu").

N-Gram kullanılmasının sebebi, Sadece kelimelere (1-gram) baksaydık, model "Kedi siyah oturdu" ile "Siyah kedi oturdu" cümlelerine aynı puanı verirdi. Oysa n-gram (özellikle 2, 3 ve 4-lü gruplar), kelimelerin sıralanmasının ve akıcılığının ne kadar doğru olduğunu ölçmemizi sağlar.

Hassasiyet (Precision) bir soruna yol açabilir. Model çok kısa ama doğru bir şey söylerse (Örneğin: Sadece "The" derse), puanı %100 çıkar. BLEU, olması gerekenden çok kısa cümleler kuran modellere ceza payı (BP) keser. Böylece modelin sadece doğru değil, aynı zamanda tam bir cümle kurması teşvik edilir.

PERPLEXİTY (Şaşkınlık)

Perplexity, bir dil modelinin bir test veri setini ne kadar iyi tahmin ettiğinin ölçüsüdür. Modelin bir sonraki kelimeyi seçerken ne kadar "şaşkın" veya "kararsız" olduğudur. Perplexity ne kadar düşükse, model o kadar iyidir. Düşük puan, modelin tahminlerinden emin olduğunu ve gerçek dil yapısına yaklaştığını gösterir.

Model, eğitim sırasında görmediği kelimelerle veya nadir gramer yapılarıyla karşılaşır olasılık düşer, dolayısıyla perplexitysi artar. Veya dili iyi öğrenemediyse, her kelimeye eşit (ve düşük) olasılık dağıtır. Bu "kararsızlık" puanı yükseltir. Perplexity eğer, artarsa model birbiriyle alakasız kelimeleri yan yana getirmeye başlar veya aynı kelimeyi tekrar kullanabilir.

Değerlendirme Kategorisi	Perplexity (PPL) Skoru ↓	Modelin Tahmin Davranışı	Akademik Başarı Seviyesi
İdeal / Teorik Sınır	1.0	Model bir sonraki kelimeyi %100 doğrulukla seçer	Kusursuz
Yüksek Performans	2-8	Model sadece en mantıklı birkaç seçenek arasında kalır	Çok Başarılı
Standart Başarı	10-25	Model Dil yapısını ve bağlamı öğrenmiştir	İyi / Yeterli
Zayıf Performans	50- 100	Model kelime seçiminde çok fazla seçenek arasında kararsızdır	Düşük Kalite
Eğitilmemiş Sistem	500+	Model dil kurallarını bilmez	Başarısız

GERÇEK DÜNYADAN ÖRNEK

En büyük örnek olarak Google translate verebiliriz.

Sanal asistanlar (Siri, Google assistant)

BASİT ÇEVİRİ MODELİ

PROJE 1

Yapılan Çalışmanın Açıklaması

Bu proje, kullanıcıdan alınan İngilizce metinleri eş zamanlı olarak Türkçe'ye çeviren etkileşimli bir makine çevirisi uygulamasıdır. Proje, Python programlama dili ve deep-translator kütüphanesi kullanılarak geliştirilmiştir. Temel amacı, karmaşık derin öğrenme mimarilerinin (RNN/LSTM tabanlı Seq2Seq) son kullanıcıya nasıl bir hizmet olarak ulaştığını göstermektir.

Kullanılan Veri Seti

Bu uygulamada yerel bir veri seti yüklemek yerine, Google'ın devasa ölçekli nöral makine çevirisi veri setleri ve önceden eğitilmiş (pre-trained) modelleri API aracılığıyla kullanılmaktadır.

Kullanılan Yöntem

Encoder: Kullanıcının girdiği İngilizce cümleyi okuyup sayısal bir bağlam vektörüne dönüştürür.

Attention: Cümle içindeki kelimeler arasındaki uzun mesafeli bağları kurarak çevirinin doğruluğunu artırır.

Decoder: Oluşturulan bu sayısal veriyi hedef dil olan Türkçe kelimelere geri çözer.

API Entegrasyonu: GoogleTranslator sınıfı kullanılarak, bulut tabanlı çalışan bu gelişmiş model ile yerel kod arasında bir köprü kurulmuştur.

Kullanılan Kodlar

- Library Import (deep_translator)
- Translator Object (GoogleTranslator(source='en', target='tr'))
- While Loop
- Error Handling: İnternet bağlantısı veya API kaynaklı olası hataların programı çökertmesini engellemek için hata yakalama blokları kullanılmıştır.

Elde Edilen Sonuçlar ve Çıktılar

```
15_2412705047.ipynb X
C:\Users\ahmet> OneDrive\ Desktop> 15_2412705047> 15_2412705047.ipynb # Kütüphaneyi indiriyorsun
% Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline --
Python 3.14.4

In [ ]:
from deep_translator import GoogleTranslator

def cevirmen_uygulaması():
    print("--- Basit Çeviri Projesi ---")
    print("Çıkmak için 'q' tuşuna basınız.\n")

    # Hedef dili seçiyoruz (Örn: İngilizce'den Türkçe'ye)
    translator = GoogleTranslator(source='en', target='tr')

    while True:
        metin = input("İngilizce metin girin: ")

        if metin.lower() == 'q':
            print("Uygulama kapatılıyor...")
            break

        try:
            # Önceden eğitilmiş sistemi kullanarak çeviri yapılıyor
            sonuc = translator.translate(metin)
            print(f"Türkçe Çeviri: {sonuc}\n")
        except Exception as e:
            print(f"❗ Hata oluştu: {e}")

# Uygulamayı başlat
cevirmen_uygulaması()

111 ✓ In 32.5s Python

Requirement already satisfied: deep-translator in c:\users\ahmet\appdata\local\programs\python\python314\lib\site-packages (1.11.4)
Requirement already satisfied: beautifulsoup4<5.0.0,>4.9.1 in c:\users\ahmet\appdata\local\programs\python\python314\lib\site-packages (from deep-translator) (4.14.3)
Requirement already satisfied: requests<3.0.0,>2.23.0 in c:\users\ahmet\appdata\local\programs\python\python314\lib\site-packages (from deep-translator) (2.31.1)
Requirement already satisfied: soupsieve>1.4.1 in c:\users\ahmet\appdata\local\programs\python\python314\lib\site-packages (from beautifulsoup4<5.0.0,>4.9.1->deep-translator) (2.8.3)
Requirement already satisfied: typing-extensions>4.0.0 in c:\users\ahmet\appdata\local\programs\python\python314\lib\site-packages (from beautifulsoup4<5.0.0,>4.9.1->deep-translator) (4.15.0)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\ahmet\appdata\local\programs\python\python314\lib\site-packages (from requests<3.0.0,>2.23.0->deep-translator) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in c:\users\ahmet\appdata\local\programs\python\python314\lib\site-packages (from requests<3.0.0,>2.23.0->deep-translator) (3.13)
Requirement already satisfied: urllib3<3,>=1.26 in c:\users\ahmet\appdata\local\programs\python\python314\lib\site-packages (from requests<3.0.0,>2.23.0->deep-translator) (2.6.3)
Requirement already satisfied: certifi>=2023.5.7 in c:\users\ahmet\appdata\local\programs\python\python314\lib\site-packages (from requests<3.0.0,>2.23.0->deep-translator) (2026.4.22)
Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 26.8.1 -> 36.1
[notice] To update, run: python.exe -m pip install --upgrade pip
--- Basit Çeviri Projesi ---
Çıkmak için 'q' tuşuna basınız.

Türkçe Çeviri: ben bir insanım

Uygulama kapatılıyor...
```

Proje 2

Yapılan Çalışmanın Açıklaması

Bu çalışma, bir yapay zeka modelinin "öğrenme" ve "hatırlama" süreçlerini simüle eden kural tabanlı bir çeviri sistemidir. Kullanıcı tarafından girilen İngilizce dizileri (sequence), sistemin belleğinde (dictionary) karşılığı olup olmadığını kontrol eder ve eşleşme varsa çıktı üretir.

Kullanılan Veri Seti

"Hello world" ve "I am going to school" gibi cümleler kodun başlangıcında sisteme manuel olarak "enjekte" edilmiştir. Sistem_ogret fonksiyonu sayesinde yeni veri girişlerine açık bir yapıdadır.

Kullanılan Yöntem

Kod, modern **Seq2Seq** mimarisinin basitleştirilmiş bir mantıksal modelini kullanır.

- **Girdi İşleme** :lower() ve strip() fonksiyonları ile kullanıcının girdisi temizlenir. Bu, NLP projelerindeki "Normalizasyon" aşamasına denk gelir.
- **Arama ve Eşleme**: Modern sistemlerdeki 'Attention' mekanizmasının çok ilkel bir hali olarak düşünülebilir; sistem hafızasındaki tüm kayıtları tarar ve girdiyle birebir örtüşen 'anahtar' bulmaya odaklanır.
- **Hafıza**: Python sözlük yapısı, bir gizli durum vektörünün içindeki bilgiyi tutması gibi, veriyi kalıcı olmasa da çalışma süresi boyunca belleğinde saklar.

Kullanılan Kodlamalar

- Dictionary (sözlük) yapısı (ogrenilmis_sistem={})
- Encapsulation (Kapsülleme)(sistem_ogret)
- Kontrol Blokları (if-else).

Elde Edilen Sonuçlar ve Çıktılar


```
15_2412705047_2.ipynb | İngilizce Metin Giriş (Press 'Enter' to confirm or 'Escape' to cancel)
C:\Users> ahmet > Desktop > 15_2412705047 > 15_2412705047_2.ipynb > PROJESİ 1: ETKİLEŞİMLİ ÖĞRENEN ÇEVİRİ SİSTEMİ
Generate + Code + Markdown | Interrupt | Restart | Clear All Outputs | Go To | Apply Variables | Outline | Python 3.114.4

# 2. ÖNCELİKLI ÖĞRETME (Hocaya göstermek için 1-2 metin)
# Kod çalıştır çalışmaz bu iki cümleyi hafızaya alacak.
sisteme_ogret("hello world", "merhaba dünya")
sisteme_ogret("I am going to school", "Okula gidiyorum")

# 3. UYGULAMA DÖNGÜSÜ (Input Aşaması)
print("--- VERELİ ÇEVİRİ SİSTEMİ BAŞLATILDI ---")
print("İpucu: 'hello world' veya 'I am going to school' yazabilirsiniz.")
print("Çıkış yapmak için 'q' tuşuna basın.\n")

while True:
    kullanıcı_girdisi = input("İngilizce Metin Girin: ")

    # Çıkış kontrolü
    if kullanıcı_girdisi.lower() == 'q':
        print("Sistem kapatılıyor...")
        break

    # Kelime/Metin kontrolü
    temiz_girdi = kullanıcı_girdisi.lower().strip()

    if temiz_girdi in öğrenilmiş_sistem:
        # Öğrenilmişse çeviriyi getir
        print(f"Çeviri: {öğrenilmiş_sistem[temiz_girdi]}\n")
    else:
        # Öğrenilmişse hata ver
        print(f"Hata: '{kullanıcı_girdisi}' henüz sisteme öğretilmedi! (Veri yok)\n")

(1) 3m55.6s Python

--- VERELİ ÇEVİRİ SİSTEMİ BAŞLATILDI ---
İpucu: 'hello world' veya 'I am going to school' yazabilirsiniz.
Çıkış yapmak için 'q' tuşuna basın.

Çeviri: merhaba dünya

Hata: 'whats up world' henüz sisteme öğretilmedi! (Veri yok)
```

SONUÇ

Genel Değerlendirme

Bu çalışma kapsamında, modern Doğal Dil İşleme dünyasının temel taşları olan **seq2seq** mimarisi ve **Attention** mekanizması hem teorik hem de pratik boyutta incelenmiştir.Yapılan analizler sonucunda, klasik RNN/LSTM tabanlı modellerin uzun dizilerdeki bilgi kaybı (bottleneck) sorununun, Attention mekanizması ile nasıl aşıldığı matematiksel ve mantıksal olarak ortaya konulmuştur.

Öğrenilenler

Bu çalışmada, değişken uzunluktaki verileri işleyebilen **Seq2Seq** mimarisinin, Encoder-Decoder yapısıyla diller arası nasıl köprü kurduğu incelenmiştir. Klasik modellerdeki bilgi kaybı probleminin, her adımda girdi dizisinin ilgili kısmına odaklanan **Attention** mekanizması ile nasıl çözüldüğü analiz edilmiştir.

Eğitim sürecini hızlandıran **Teacher Forcing** ve en mantıklı çıktı dizisini seçen **Beam Search** gibi stratejilerin çalışma prensipleri öğrenilmiştir. Ayrıca, model başarısını ölçen **BLEU skoru** ve belirsizliği ifade eden **Perplexity** metrikleri matematiksel olarak ele alınmıştır. Yapılan Python uygulamalarıyla, bu teorik temellerin çalışan birer çeviri sistemine dönüştürülme süreci deneyimlenmiştir.

Akademik Kaynakça

- [1] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proc. Advances in Neural Information Processing Systems (NIPS)*, Montreal, QC, Canada, 2014, pp. 3104–3112.
- [2] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, 2015.
- [3] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Doha, Qatar, 2014, pp. 1724–1734.
- [4] A. Vaswani et al., "Attention is all you need," in *Proc. Advances in Neural Information Processing Systems (NIPS)*, Long Beach, CA, USA, 2017, pp. 5998–6008.
- [5] K. Papineni, S. Roukos, T. Ward, and W. J. Zhu, "BLEU: a method for automatic evaluation of machine translation," in *Proc. 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, Philadelphia, PA, USA, 2002, pp. 311–318.
- [6] M. T. Luong, H. Pham, and C. D. Manning, "Effective approaches to attention-based neural machine translation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Lisbon, Portugal, 2015, pp. 1412–1421.
- [7] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. draft, Upper Saddle River, NJ, USA: Pearson, 2023.
- [8] R. J. Williams and D. Zipser, "A learning algorithm for continually running fully recurrent neural networks," *Neural Computation*, vol. 1, no. 2, pp. 270–280, Jun. 1989.